In [28]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import confusion_matrix, accuracy_score
```

In [29]:
```
# pip install tensorflow ignore if already installed
```

In [30]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

In [31]:
```
df = pd.read_csv("Churn_Modelling.csv")
```

In [32]:
```
df.head()
```

Out[32]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1

In [33]:
```
# 2. Distinguish the feature and target set
X = df.iloc[:, 3:13]   # Features (from CreditScore to EstimatedSalary)
y = df.iloc[:, 13]     # Target (Exited column)
```

In [34]:
```
# Encoding categorical variables (Geography and Gender)
labelencoder_gender = LabelEncoder()
X['Gender'] = labelencoder_gender.fit_transform(X['Gender'])


# One-hot encode Geography
X = pd.get_dummies(X, columns=['Geography'], drop_first=True)
```

In [36]:
```
# 3. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In [37]:
```
# Normalize the data
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [47]:
```
X_train_df = pd.DataFrame(X_train, columns=X.columns)

print("Sample of normalized training data:")
display(X_train_df.head())

print("\nMean of features after scaling:\n", X_train_df.mean())
print("\nStandard deviation of features after scaling:\n", X_train_df.std())
```

Sample of normalized training data:

	CreditScore	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Geogra
0	0.356500	0.913248	-0.655786	0.345680	-1.218471	0.808436	0.649203	0.974817	1.367670	
1	-0.203898	0.913248	0.294938	-0.348369	0.696838	0.808436	0.649203	0.974817	1.661254	
2	-0.961472	0.913248	-1.416365	-0.695393	0.618629	-0.916688	0.649203	-1.025834	-0.252807	
3	-0.940717	-1.094993	-1.131148	1.386753	0.953212	-0.916688	0.649203	-1.025834	0.915393	
4	-1.397337	0.913248	1.625953	1.386753	1.057449	-0.916688	-1.540351	-1.025834	-1.059600	

```
Mean of features after scaling:
 CreditScore          5.435652e-16
Gender               2.842171e-17
Age                 -1.896261e-16
Tenure               6.661338e-17
Balance             -3.099743e-16
NumOfProducts        1.065814e-16
HasCrCard           -3.197442e-17
IsActiveMember      -1.776357e-17
EstimatedSalary      1.776357e-17
Geography_Germany   -4.263256e-17
Geography_Spain     -7.194245e-17
dtype: float64

Standard deviation of features after scaling:
 CreditScore          1.000063
Gender               1.000063
Age                  1.000063
Tenure               1.000063
Balance              1.000063
NumOfProducts        1.000063
HasCrCard            1.000063
IsActiveMember       1.000063
EstimatedSalary      1.000063
Geography_Germany    1.000063
Geography_Spain      1.000063
dtype: float64
```

In [38]:
```python
# 4. Initialize and build the Neural Network model
model = Sequential()
model.add(Dense(units=6, activation='relu', input_dim=X_train.shape[1]))
model.add(Dropout(0.3))
model.add(Dense(units=6, activation='relu'))
model.add(Dense(units=1, activation='sigmoid'))
```

```
C:\Users\vishw\AppData\Roaming\Python\Python313\site-packages\keras\src\layers\core\dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [39]:
```python
# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

We are treating countries with ordinal values(0 < 1 < 2) but they are incomparable. To solve this we can use one hot encoding. We will perform some standardization

In [40]:
```python
# Train for 20 epochs
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test, y_test))
```

```
Epoch 1/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.4198 - loss: 0.8179 - val_accuracy: 0.7720 - val_loss: 0.5532
Epoch 2/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7772 - loss: 0.5545 - val_accuracy: 0.8030 - val_loss: 0.4643
Epoch 3/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7972 - loss: 0.4954 - val_accuracy: 0.8010 - val_loss: 0.4377
Epoch 4/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8008 - loss: 0.4648 - val_accuracy: 0.8045 - val_loss: 0.4268
Epoch 5/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8011 - loss: 0.4573 - val_accuracy: 0.8065 - val_loss: 0.4175
Epoch 6/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8078 - loss: 0.4490 - val_accuracy: 0.8195 - val_loss: 0.4079
Epoch 7/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8127 - loss: 0.4361 - val_accuracy: 0.8270 - val_loss: 0.3998
Epoch 8/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8255 - loss: 0.4216 - val_accuracy: 0.8305 - val_loss: 0.3903
Epoch 9/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8236 - loss: 0.4171 - val_accuracy: 0.8340 - val_loss: 0.3804
Epoch 10/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8194 - loss: 0.4191 - val_accuracy: 0.8370 - val_loss: 0.3727
Epoch 11/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8226 - loss: 0.4088 - val_accuracy: 0.8430 - val_loss: 0.3699
Epoch 12/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8337 - loss: 0.3952 - val_accuracy: 0.8435 - val_loss: 0.3662
Epoch 13/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8224 - loss: 0.4089 - val_accuracy: 0.8485 - val_loss: 0.3653
Epoch 14/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8277 - loss: 0.4097 - val_accuracy: 0.8500 - val_loss: 0.3643
Epoch 15/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8297 - loss: 0.3984 - val_accuracy: 0.8500 - val_loss: 0.3627
Epoch 16/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8222 - loss: 0.4016 - val_accuracy: 0.8505 - val_loss: 0.3617
Epoch 17/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8369 - loss: 0.3958 - val_accuracy: 0.8495 - val_loss: 0.3608
Epoch 18/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8271 - loss: 0.3976 - val_accuracy: 0.8495 - val_loss: 0.3595
Epoch 19/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8279 - loss: 0.4004 - val_accuracy: 0.8485 - val_loss: 0.3607
Epoch 20/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8293 - loss: 0.3914 - val_accuracy: 0.8490 - val_loss: 0.3599
```

In [53]:
```python
# Print accuracy per epoch
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

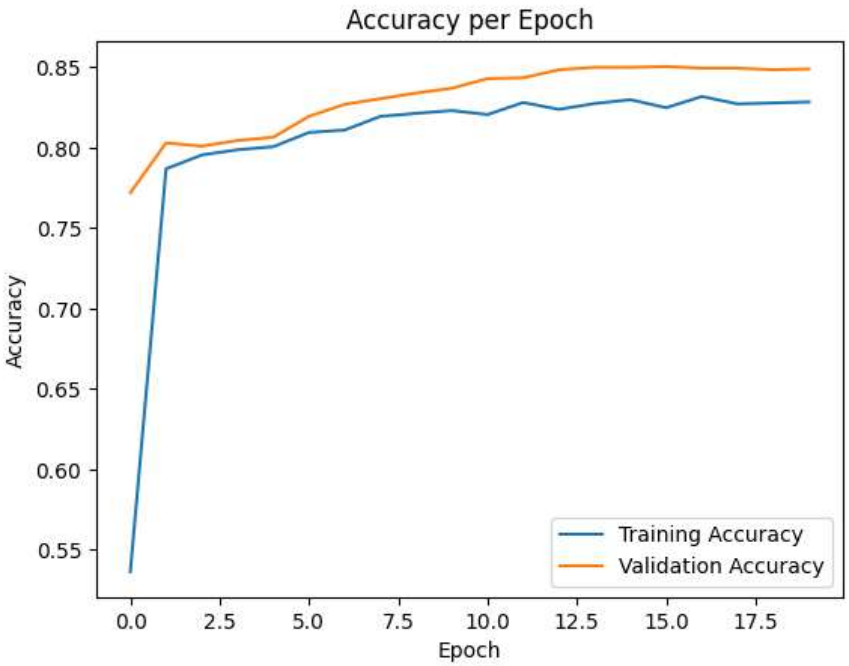
print("Accuracy per Epoch:\n")
for i in range(len(train_acc)):
    print(f"Epoch {i+1}: Training Accuracy = {train_acc[i]*100:.2f}%, Validation Accuracy = {val_acc[i]*100:.2f}%")
```

```
Accuracy per Epoch:

Epoch 1: Training Accuracy = 53.64%, Validation Accuracy = 77.20%
Epoch 2: Training Accuracy = 78.69%, Validation Accuracy = 80.30%
Epoch 3: Training Accuracy = 79.55%, Validation Accuracy = 80.10%
Epoch 4: Training Accuracy = 79.87%, Validation Accuracy = 80.45%
Epoch 5: Training Accuracy = 80.06%, Validation Accuracy = 80.65%
Epoch 6: Training Accuracy = 80.95%, Validation Accuracy = 81.95%
Epoch 7: Training Accuracy = 81.10%, Validation Accuracy = 82.70%
Epoch 8: Training Accuracy = 81.95%, Validation Accuracy = 83.05%
Epoch 9: Training Accuracy = 82.14%, Validation Accuracy = 83.40%
Epoch 10: Training Accuracy = 82.31%, Validation Accuracy = 83.70%
Epoch 11: Training Accuracy = 82.06%, Validation Accuracy = 84.30%
Epoch 12: Training Accuracy = 82.81%, Validation Accuracy = 84.35%
Epoch 13: Training Accuracy = 82.39%, Validation Accuracy = 84.85%
Epoch 14: Training Accuracy = 82.75%, Validation Accuracy = 85.00%
Epoch 15: Training Accuracy = 82.99%, Validation Accuracy = 85.00%
Epoch 16: Training Accuracy = 82.49%, Validation Accuracy = 85.05%
Epoch 17: Training Accuracy = 83.19%, Validation Accuracy = 84.95%
Epoch 18: Training Accuracy = 82.73%, Validation Accuracy = 84.95%
Epoch 19: Training Accuracy = 82.79%, Validation Accuracy = 84.85%
Epoch 20: Training Accuracy = 82.85%, Validation Accuracy = 84.90%
```

In [43]:
```python
import matplotlib.pyplot as plt
```

In [44]:
```
# Accuracy per epoch
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



In [51]:
```
acc = accuracy_score(y_test, y_pred)
print("Final Accuracy: {:.2f}%".format(acc * 100))
```

```
Final Accuracy: 84.90%
```

In [52]:
```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[1557   50]
 [ 252  141]]
```